

TravelGo!

Team 11

Aník van Deursen, Riya Gupta, Radha Mujumdar, Diana Todoran

GitLab Repository: [<https://gitlab.ewi.tudelft.nl/cs4505/2025-2026/team-11>]
(<https://gitlab.ewi.tudelft.nl/cs4505/2025-2026/team-11>)

1 Introduction

1.1 Problem Statement

Traveling is widely regarded as an enriching experience that allows individuals to explore new places, engage with diverse cultures, and participate in the social life of a destination. However, travelers often face considerable challenges. Limited time can make it difficult to fully engage with the abundance of culture and history, and often makes it impossible to visit all tourist attractions.

Another challenge lies in connecting with other travelers. Tourist, especially those traveling alone, can face difficulties in finding like-minded individuals. Because of this they might not create meaningful connections during their travels, which can limit the overall enjoyment of their experience.

1.2 Context & purpose

To address these challenges, we want to create an interactive digital platform that makes traveling easier and more enjoyable for both group and solo travelers across various countries: TravelGo. Central to the platform is a map with all of the tourist attractions, including hidden spots. The platform aims to enrich cultural engagement with competitive games (for example quiz competitions) with a reward system through leaderboards. To make this competition enjoyable for both inexperienced and experienced travelers, the platform uses a rating system with leagues. Users can additionally earn points by going to an attraction and thus ticking it off their bucket list. These points can be used to for example get a discount on entry fees or to get an exclusive souvenir.

Another main part of TravelGo is making it easier for travelers to engage with each other. To encourage tourist interaction, the platform will include a chatroom. This chatroom helps travelers to meet new people that are at the same location. Users can also post new or hidden attractions that they recommend to others. Because of this, tourist will know which attractions will be the most worthwhile to visit.

2 Wardley Map

Building on the system's vision and context, the Wardley Map highlights how TravelGo blends innovation with open source or standardized solutions.

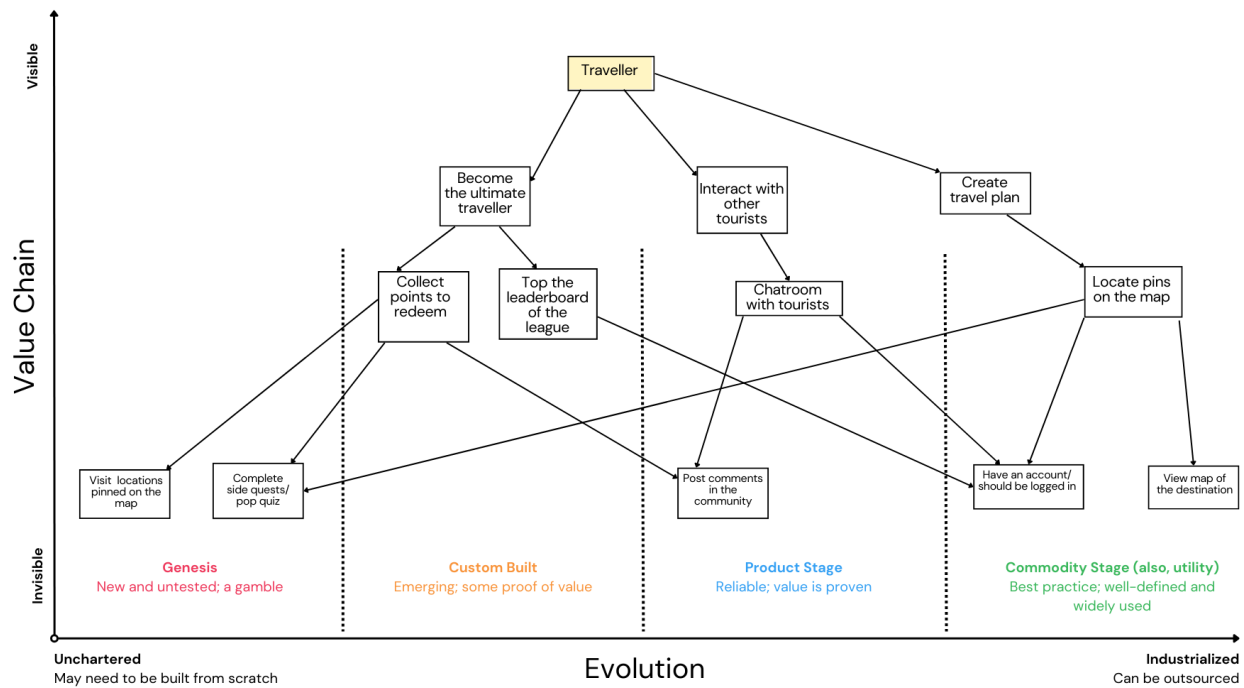


Figure 2.1 : Wardley map

Genesis : This space contains novel, experimental features like side quests, cultural quizzes and souvenir based rewards which are not yet mainstream in the travel tech domain. They provide differentiation, but also present a high risk of adoption and design.

Custom Built: Features like leaderboards and points system are placed here. While interactive platforms are popular in other domains like fitness and education, applying it specifically to cultural travel remains relatively bespoke. These features distinguish TravelGo from commodity travel apps but are less risky than Genesis elements.

Product Stage: More well known features such as community chatrooms and posting comments fall under this category. These are standard capabilities available in many social or booking apps, but TravelGo customizes them for cultural travel contexts. They are visible to users but do not offer radical innovation.

Commodity Stage: Underlying infrastructure such as digital maps, location pinning, user accounts and APIs are considered commodity. They are invisible to end-users and widely available through third-party providers like Google Maps. TravelGo does not attempt to innovate here but instead rely on stable, low-cost services.

3 Market Research

To identify opportunities for innovation, it is useful to examine the existing players in the travel tech market. The table below presents a comparative analysis of these competitors, outlining their strengths, weaknesses, and potential gaps.

Serial No.	Competitor	Strengths	Weaknesses	Opportunity for us
------------	------------	-----------	------------	--------------------

Serial No.	Competitor	Strengths	Weaknesses	Opportunity for us
1	TripRanger	Gamifies travel, fun challenges	Limited user base, not mainstream, focus mostly on gamification rather than deep cultural immersion	We can combine interactivity and cultural depth, making it more meaningful
2	Polarsteps	Very popular, creates a visual travel journal, great for reliving your travel experiences. Helps share trips with friends and family	Mostly passive (recording, not interacting); limited social engagement beyond sharing	We add interactivity, side quests, community building
3	Withlocals	Popular; focused on authentic cultural experiences by connecting with locals/guides	Paid/expensive; depends heavily on guide availability; no gamification or community	Make authentic culture accessible and community driven
4	Viator	Huge catalog of tours & activities	Overly commercial and not personalized, lacks interactive features	Focus on personalized, fun challenges and cultural immersion rather than generic tours
5	Komoot	Expertise in outdoor navigation and route planning	Very niche (outdoors only)	Add cultural challenges with a social community
6	Mapstr	Save and share favorite places easily	More like a utility app (maps), lacks engagement, no gamification	Build on map sharing, but with interactive quests, culture and competition
7	Google Maps	Massively popular for navigation, locating places	Lacks the interactive aspect	We add interactivity and cultural immersion

Table 3.1 : Competitor Analysis

3.1 Key Insights

Is there demand?

Market trends indicate that travellers increasingly seek experiences over mere sightseeing. Therefore, platforms that are interactive and community-driven are especially popular among younger travellers

nowadays.[\[2\]](#)[\[3\]](#)[\[4\]](#)

What's missing in current solutions?

- Most travel apps focus on either utility (planning, booking) or authenticity (guided tours).
- Few apps combine an interactive, culturally immersive experience with community building.
- Existing solutions often lack interactivity, offline usability, or gamified side quests.

How TravelGo fills that gap

- Integrates interactive, culturally immersive experiences with community engagement into a single platform.
- Enables users to complete side quests, join challenges, and earn rewards while exploring off-beat locations.
- Promotes social discovery by connecting travellers nearby.
- Built with an offline-first architecture, considering real traveller needs such as spotty internet and battery constraints.

4 Challenges

I. Data Privacy & Trust : Data privacy and trust are critical concerns for TravelGo, as the platform collects personal information such as names, locations, and travel history. Mishandling this data could result in legal issues and a loss of customer trust. However, several ambiguities arise, such as determining how much data should be collected, finding the right balance between personalization and privacy, and managing cross-border compliance. From an architectural perspective, the system must implement data minimization by collecting only what is necessary, incorporate strong consent management, and ensure secure data storage and transmission. Additionally, regional data hosting may be required to meet compliance regulations.

II. Cultural Sensitivity & Representation : TravelGo promotes cultural quests and hidden gems, but what one traveler considers a "hidden gem" could, in fact, be a sacred site or a sensitive local tradition. Such misrepresentation risks community backlash and even legal consequences. This raises ambiguities around who decides what is "authentic" enough to feature and how to avoid cultural appropriation or trivialization. Architecturally, the platform may require robust content vetting workflows, potentially involving local validators or approval pipelines for sensitive submissions. Flexible data models will also be needed to support metadata tagging of cultural content such as marking items as sensitive, sacred, or family-friendly, along with the ability to enable regional customization where necessary.

III. Technical Constraints : Travelers often face unreliable connectivity, yet TravelGo's core features such as locating pins, chat, and quests may rely heavily on online services. This raises ambiguities around which features should remain functional offline and how much data can reasonably be cached on a device without draining storage or battery. Through an architectural lens, the system may need to adopt an offline-first design, ensuring that critical functions remain usable without constant connectivity. In particular, GPS tracking without internet access would depend on operating system support and offline map APIs.

IV. Community Moderation & Safety : TravelGo's chatrooms and commenting features are central to its social experience, but like most online community spaces, they can quickly become targets for spam, harassment, scams, or inappropriate content. This introduces ambiguities around whether moderation should rely on AI and keyword blocking or be led by human moderators, how to enforce community rules across

diverse cultures and languages, and whether moderation should be centralized or distributed. From a design point of view, this creates the need for scalable content moderation pipelines, along with careful handling of flagged content since its storage and processing bring compliance and legal liabilities. At the same time, balancing low latency for real-time chat with effective filtering poses significant technical challenges.

V. Ecosystem Dependencies : TravelGo relies on third-party APIs for critical functionalities such as maps and login, but this dependency introduces several uncertainties. Providers may change pricing models, discontinue services, or have limited availability in certain regions. This raises the question of whether to design for multi-provider fallback or accept the risks of locking into a single provider. From an architectural standpoint, it becomes important to introduce abstraction layers rather than hardcoding to any one API, consider vendor diversity to reduce reliance on a single source, and actively monitor latency and reliability across providers to ensure a consistent user experience.

5 Stakeholders

Travellers create demand by seeking personalisation and meaningful connections, gravitating toward the cultural, recreational, and community-driven offerings available on the platform. They engage with attraction sites and redeem discounts at local businesses, boosting visitor numbers and generating higher revenue for these stakeholders while promoting their trade. Management works closely with regulatory authorities and tourism boards to ensure compliance and safety. Strengthening the platform's visibility, travel communities and influencers amplify positive experiences through information sharing and marketing. Together, TravelGo delivers reliable, user-centric travel solutions while balancing the needs of customers, local communities, and the broader ecosystem.

5.1 Primary Stakeholders

- Tourists and Travellers
 - Main users of the platform
 - Engage in competitive games, meet like-minded individuals and be interested in cultural enrichment
- Attraction Sites
 - Higher footfalls and visitor engagement resulting in an increase in ticket sales
- Local Businesses
 - Advertisements and partnerships will gain more exposure, eliciting profits
 - Support special offers and discounts

5.2 Secondary Stakeholders

- Government and Tourism Boards
 - Boost in tourism while ensuring compliance of the local laws
 - Control the digital platforms, tourism, and safeguarding the cultural
- Local Communities
 - Cultural representatives who gain from more interactions and business from tourists
 - Might be worried about cultural sensitivity
- Investors and Sponsors
 - Support financially and expecting returns in the form of partnerships, advertisements, subscriptions, and even positive publicity

5.3 Tertiary Stakeholders

- Travel Agencies and Tour Operators
 - Competitors or can be potential partners
- Online Travel Communities and Influencers
 - Increase online engagement and positive word of mouth through social media

The power/interest grid is used to classify stakeholders according to their influence and level of engagement. High-power, high-interest stakeholders such as travellers and attraction owners are closely managed since they are the core users. Tourism boards and regulators have high power but lower day-to-day interest, requiring consultation occasionally. Communities, influencers, and competitors have lower power but varying levels of interest, monitored for promotion and market positioning.

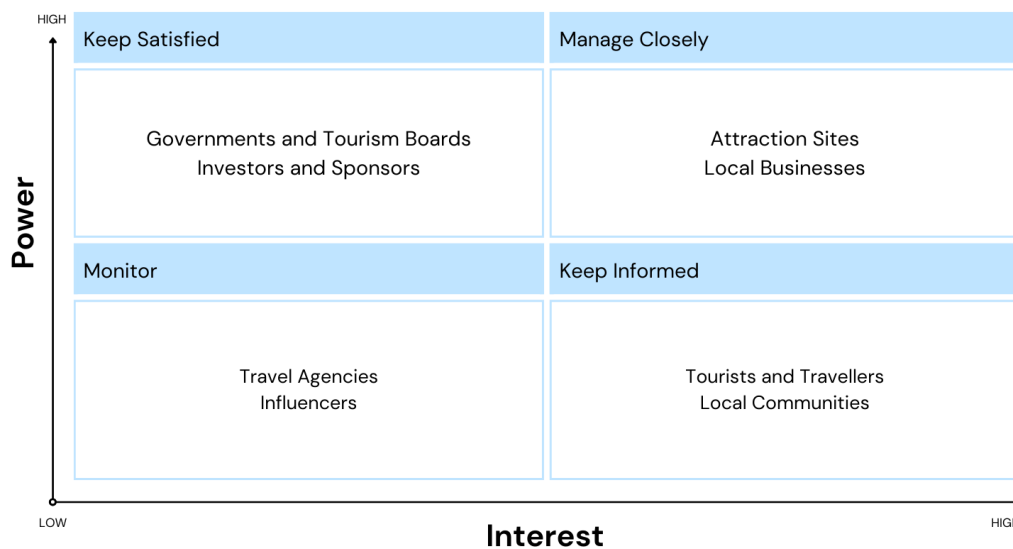


Figure 5.1: Power / Interest Grid

6 Features

Features are distinct pieces of functionality that deliver value to users. They define what the software can do and are often used to plan and develop the product throughout its lifecycle.

Feature ID	Feature Description
F1	Maintain an individual user bucket-list of attractions.
F2	Display a map with available countries and available attractions in each country.
F3	Maintain a community chatroom for attraction discoveries and user interactions.
F4	Display advertisements.

Feature ID	Feature Description
F5	Allow for premium features. (Let the user know when there are discounts for accommodation)
F6	Keep track of the user's daily quiz score and their placement in the leaderboard.
F7	Reward the top competitors of the leaderboard with discounts, vouchers or free souvenirs.

Table 6.1: List of features that will be implemented for the final product.

7 Use Case Scenarios

Use case scenarios explain how a user works with a system to accomplish certain tasks or objectives. They outline the steps needed to achieve a set objective and also help define system requirements, derived from the user stories, which can be found in the Appendix A.

Use Case ID	Use Case Description
UC1	Automatically display a map of available countries.
UC2	Automatically display a map of available attractions for the selected country.
UC3	Create a traveling plan for a given number of visiting days.
UC4	Create a sharable link of the traveling plan.
UC5	Allow users to cross-off visited attractions from the plan/bucket-list.
UC6	In the case of quizzes, the platform must offer a selection of questions, weighted in points. The summed points increase the user's daily score for the leaderboard.
UC7	Determine the user's reward based on their score.
UC8	Display advertisements for internal and external sponsors.
UC9	Display available discounts for internal and external sponsors.
UC10	Process shop fee for earned souvenir by the user.
UC11	Maintain a hidden gem list of attractions.
UC12	List various restaurants or outdoors environments for nearby attractions.
UC13	(In case of premium) Process subscription fee for unlocking premium for the user's account.
UC14	(In case of premium) display a list of available accommodations.
UC15	(In case of premium) Remove advertisements and any sort of advertisement.

Table 7.1: List of use case scenarios that will be implemented for the final product.

8 Requirements

8.1 Functional Requirements

- Interactive Map and Destination Discovery
 - The system shall display an interactive world map showing available countries and their major attractions.
 - Upon selecting a country, users should see recommended attractions, local experiences, and “hidden gems” contributed by communities and tourism boards.
 - The map should be dynamic, supporting zoom, filter, and search functionalities to improve user exploration and engagement.
- Gamification and Leaderboards
 - The system shall include daily quizzes or travel challenges that award points based on correct answers and engagement.
 - User points shall contribute to a leaderboard system that ranks participants daily or weekly.
 - Rewards such as discounts, badges, or vouchers shall be distributed based on user achievements and cumulative points.
- Reward Management and Marketplace Integration
 - The platform shall manage a reward system where users can redeem earned points for souvenirs or travel discounts.
 - It shall handle secure payment processing for shop purchases or reward redemptions.
 - Users should be able to view available rewards, track redemptions, and access partner offers directly within the platform.
- Sponsorships, Ads, and Discounts
 - The system shall display sponsored advertisements from internal and external partners, relevant to user interests and destinations.
 - It shall present discounts, offers, and promotions from local businesses, tourism boards, and sponsors in a non-intrusive manner.
 - The platform shall provide a premium subscription that removes all advertisements for an ad-free experience.
- Premium Subscription and Content Access
 - Premium users shall have access to exclusive features, including accommodation listings, advanced travel recommendations, and enhanced itinerary options.
 - The system shall support secure subscription fee processing and manage user entitlements based on their subscription tier.
- Local Community and Business Integration
 - The platform shall highlight local restaurants, outdoor venues, and small businesses near attractions.
 - It shall feature a “Hidden Gems” section, showcasing lesser-known attractions curated by locals.
 - Local communities and tourism boards shall be able to submit or update listings for validation by the system.

- Data Accuracy and Content Management
 - The platform shall ensure that all displayed data (maps, attractions, reviews, offers) are accurate and periodically verified.
 - The system shall maintain data integrity and prevent unauthorized modifications to public information.
- Communication and Social Interaction
 - The system shall include a community forum or chat feature to facilitate discussions between travellers, influencers, and local hosts.
 - Users shall be able to exchange travel tips, share experiences, and connect with like-minded explorers.
- System Integration and Interoperability
 - All services (map, user, post, leaderboard, rewards, chat, etc.) shall interact through an API Gateway using event-driven communication.
 - Each service shall be loosely coupled, supporting independent deployment and scalability.
 - The platform shall ensure data confidentiality and integrity across services via secure communication protocols.

8.2 Non-Functional Requirements

Non-functional requirements aim to outline how a system should perform when completing its tasks, based on the system's implementation. According to Sethi, R. (2023) [11], non-functional requirements are better known as quality attributes as they describe the system's performance rather than its functionality. These specific attributes for TravelGo are further elaborated in the following section.

9 Quality Attributes

Quality attributes describe desirable properties of a system. For creating TravelGo we want to consider the following elements:

Modularity: Since the platform will have distributed deployment, TravelGo needs to use modules. Modularity is also important for parallel development and incrementally building.

Performance: TravelGo should not take a long time to respond, since this will annoy users and might make them not want to use the app. We also want the system to be able to take many requests at the same time.

Scalability: At first, TravelGo will not have a lot of users. Of course we hope to increase this amount over time. Because of this, the architecture should be designed for growth.

Availability: The system should be functioning correctly 24/7, since our users are in many different timezones, which means that the platform is always in use. We also want the platform to be partially available when users are not connected to the internet, which happens often when traveling.

Authentication: Because users can buy a premium subscription that unlocks features, we need to confirm their identity.

Integrity: We need to be certain that all information on the platform is authentic, trustworthy and cannot be tampered with. It is also important that companies are paid for their services, and users get access to the additional features if they purchase the premium subscription.

Confidentiality: Since we work with our users' and companies' personal data and have a chatroom functionality, we need to be certain that this sensitive data is not leaked or shared.

Adaptability: It is important that our system is extensible, because we want to add new features as the platform becomes more successful. We also want our system to be modifiable, so that the implemented functionality can be changed if needed and we can remove less successful features.

Portability: We want our platform to be available for all systems. If we make the app portable, we can save a lot of costs and effort.

The quality attributes that we primarily want to focus on are **scalability, modularity, confidentiality** and **integrity**.

9.1 Trade-offs

For the main quality attributes, there are some trade-offs that we should keep in mind while designing the system:

Scalability vs Integrity: TravelGo should be designed for growth. However, a large user-base all around the world could have an impact on integrity. For example, with more tourist attractions, it might be more difficult to assure that all the information on the platform is authentic and trustworthy.

Scalability vs Performance: We want TravelGo to have as many users as possible. Having said that, with more users the performance of the app might go down, especially since the users will be from all around the world.

Modularity vs Performance: If the system is designed to be modular, the performance of the app might be worse. This is the case because the modules have to communicate with each other, which may cause delays.

Confidentiality vs Modularity: Confidentiality and modularity are both very important for most of our stakeholders. However, if the system is designed to be modular, it is harder to secure our system, since there will be multiple communication points between the modules.

9.2 Important quality attributes for stakeholders

Different stakeholders have different reasoning why certain quality attributes are the most important. This is shown in table 10.2.

Quality Attribute	Expectation	Stakeholders
Scalability	- System should be able to scale to support peak tourist seasons	- Governments & Tourism Boards

Quality Attribute	Expectation	Stakeholders
	- The system should be capable of displaying all attraction sites	- Attraction Sites
	- System should recommend my business when planning a trip without being influenced by the user's current location	- Local Businesses - Investors & Sponsors
	- Communities would want to grow without the app slowing down, from a handful of members to thousands	- Travel Communities & Influencers
	- System should scale for multiple users exploring community resources	- Local Communities
	- System should be fast and easily understandable even when multiple users are utilizing the app or when multiple leagues are run for different countries	- Tourists and Travellers
Modularity	- System should include the attraction site in all features, new and old	- Attraction Sites
	- System should be able to integrate new features seamlessly	- Travel Communities & Influencers - Governments & Tourism Boards - Local Communities
Integrity	- System should be reliable such that users can trust it and companies want to collaborate with it	- Governments & Tourism Boards
	- If I pay for a partnership, advertisement, subscription or positive publicity, I must get what I paid for	- Investors & Sponsors
	- The system should offer compensation for my service before advertising on the platform	- Local Businesses - Attraction Sites

Quality Attribute	Expectation	Stakeholders
	- All the information on the platform must be authentic and trustworthy	- Travel Communities & Influencers - Tourists and Travellers - Local Communities
	- System should provide advertised features if the premium subscription is paid for	- Tourists and Travellers
	- System should reward a winner from leagues. In the case of equal scores, alternative solutions must be implemented	- Tourists and Travellers
Confidentiality	- Sensitive data should not be leaked or shared	- Investors & Sponsors - Attraction Sites - Local Businesses - Tourists and Travellers

Table 9.2: Table with reasoning why certain quality attributes are important to stakeholders

10 Architecture Design

Now that the context of the system has been defined, the next step is to determine the architectural design. This will be based on our main quality attributes: scalability, modularity, integrity and confidentiality.

10.1 Architectural Styles

To make sure that the system meets its most important quality attributes, an appropriate architectural style has to be selected. We will consider the following architectures: the monolithic architecture, the microkernel architecture, the serverless architecture and the microservices architecture.

10.1.1 Monolithic Architecture

The monolithic architecture [1] mainly has disadvantages for our system. For two of our most important quality attributes, modularity and scalability, the monolithic architecture is a poor choice. In terms of modularity, The system cannot be distributed, and deployment in the cloud is very expensive. Moreover, if one thing does not work the rest does not work either. Since our system has many functionalities, this is not ideal, because if one function does not work all of the other functionalities do not work either. Additionally, if you change one thing in the monolith, everything has to be rebuild. Since there will be many changes on the platform, this is would be difficult. Tourist attractions will be added and removed all the time. Regarding

scalability, the existing code cannot be reused. This means that if we want to use part of our existing code, we would have to reuse all of the code of our monolith, even the parts that we do not need.

However, the monolithic architecture has one advantage: it might be easier to secure, since it has fewer communication points. This could have a positive impact on the confidentiality and integrity of the system.

10.1.2 Microkernel Architecture

Microkernel architecture will provide the platform with a lightweight and stable core while allowing the features to be added as independent plug-ins. This modularity will make it easier to expand and maintain the system, ensuring that failures in one module do not affect the entire platform. However, the platform relies heavily on high-performance interactions with external systems like map services and tourism boards and live community interaction through chats and posts. A microkernel has a high performance overhead due to the constant communication between the core and plug-ins, which could degrade user experience during peak usage. Additionally, it also increases the development time and cost of designing and maintaining interfaces between the kernel and multiple modules.

10.1.3 Serverless Architecture

Serverless architecture enables the platform to be scaled automatically based on user demand and reduces the need for managing servers by developer since cloud providers manage it [5]. This architecture allows platforms to simplify deployment and improves operational efficiency. However, TravelGo would heavily rely on real-time interactions such as leaderboards, competitive quizzes, and user engagement based on location. Therefore, even though serverless systems are highly scalable and stateless by nature, it can lead to more overhead in terms of response time and cost when handling large number of complex processes. Furthermore, it implies less control on the server side which could breach confidentiality and security protocols.

10.1.4 Microservices Architecture

Scalability: TravelGo will serve a diverse user base with travelers from around the world. Microservices can be deployed across multiple regions, closer to where users are, which reduces latency and ensures a seamless experience globally. Since each service can scale independently, TravelGo can easily handle spikes in usage, for example, scaling the chatroom service during travel seasons without affecting payments or recommendations.

Modularity: TravelGo offers a variety of features like community interactions, cultural content, side quests, and plans to add even more in the future. Microservices naturally support modularity by dividing the system into small, loosely coupled services, each built around a specific business capability. This allows TravelGo to implement, test, and deploy new features independently, without slowing down the rest of the system. For instance, a new "local quiz" module could be rolled out without touching the existing leaderboard or chat functionality.

Integrity: Because TravelGo collects sensitive personal data such as names, payment details, and location information, integrity and security are critical. Microservices help here by isolating services and containing failures. If one service is compromised, others remain unaffected, protecting the majority of user data. While microservices do increase the number of potential entry points into the system, these risks can be managed with event-driven communication, strong authentication, and encryption. This ensures that TravelGo maintains user trust by preserving data security and system reliability.

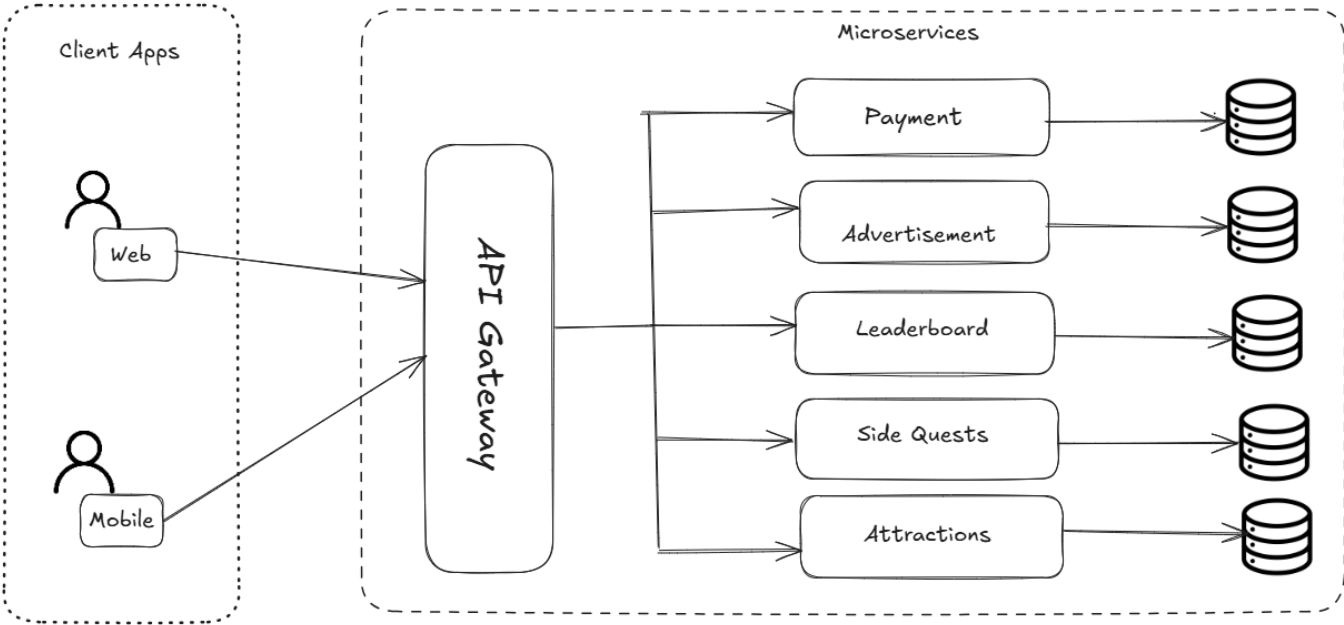


Figure 10.1.4: Microservices architecture of the system

10.1.5 Trade-Off Analysis for Architectural Styles

Although all of the four architectural styles have their disadvantages and advantages, as shown in Table 11.1.5, a single approach must be selected. The monolithic architecture is the least suitable for our system, since it has significant disadvantages in terms of modularity and scalability. The microkernel and serverless architectures are both reasonable options, however they have less important advantages and more disadvantages compared to the microservice architecture.

The microservice architecture has advantages for all of our main quality attributes. The only disadvantage is that it increases the number of potential entry points into the system, which can be managed using event-driven communication. Therefore, the **microservice architecture** is the most advantageous for our system.

Advantages		Disadvantages
Monolithic	- Easier to secure	- System cannot be distributed
		- Deployment in cloud is expensive
Microkernel	- Allows features to be added as plug-ins	- If one thing does not work the rest does not either
	- Failures in one module does not affect others	- If one thing is changed everything has to be rebuild
Serverless	- Automatic scaling	- Existing code cannot be reused
	- Simplifies deployment	
	- Improves operational efficiency	- High performance overhead
		- Increases development time & cost of designing and maintaining
		- More overhead in response time and cost
		- Less control on server side

	Advantages	Disadvantages
Microservices	- Each service can scale independently - Features can be implemented, tested and deployed independently - Isolates services and contains failures	- Number of potential entry points into the system increases

Table 10.1.5: Advantages and disadvantages of the architectural styles

10.2 Architectural Views

The C4 model [9] presents are a set of diagrams used to visualize the architecture of a software architecture at different levels of detail. Together, these views help communicate clearly to stakeholders how a system fits into its environment, how it’s structured internally, and how its parts interact.

10.2.1 Context View

The System Context Diagram highlights TravelGo’s role within its environment. It shows the platform as the central system interacting with travellers, attraction owners, and several external systems such as map providers, tourism boards, influencers, and competitors. The diagram illustrates key flows of information (e.g. travellers providing personal information, owners submitting attractions, the platform requesting maps) and helps define clear system boundaries and dependencies.

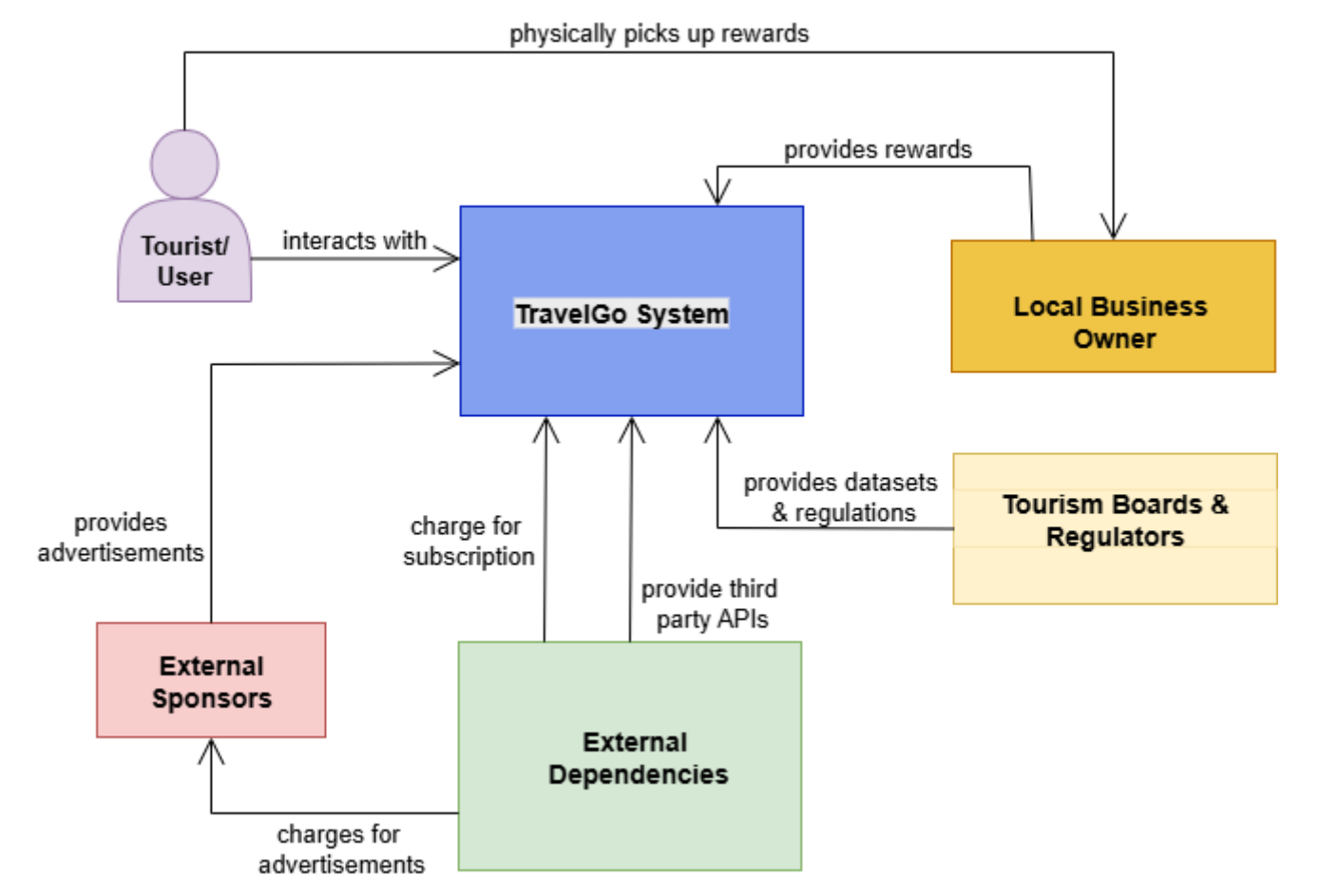


Figure 10.2.1: Context Diagram

10.2.2 Container View

The next view depicts all parts from the context view further elaborated into containers. As you can see in Figure 10.2.2 below, the TravelGo System, Local Business Owners, Tourism Boards and External Dependencies now showcase more details about their inner workings.

The system's core container is the App, which interacts with the Tourist and utilizes internal databases to manage user data, as well as external databases to interact with the Tourism Boards. Furthermore, the connection with the External Dependencies is carried out through 2 connections between the Payment System for processing user subscription and the Third-Party Map API. Lastly, the Local Businesses contain the Rewards and the functional relation is presented.

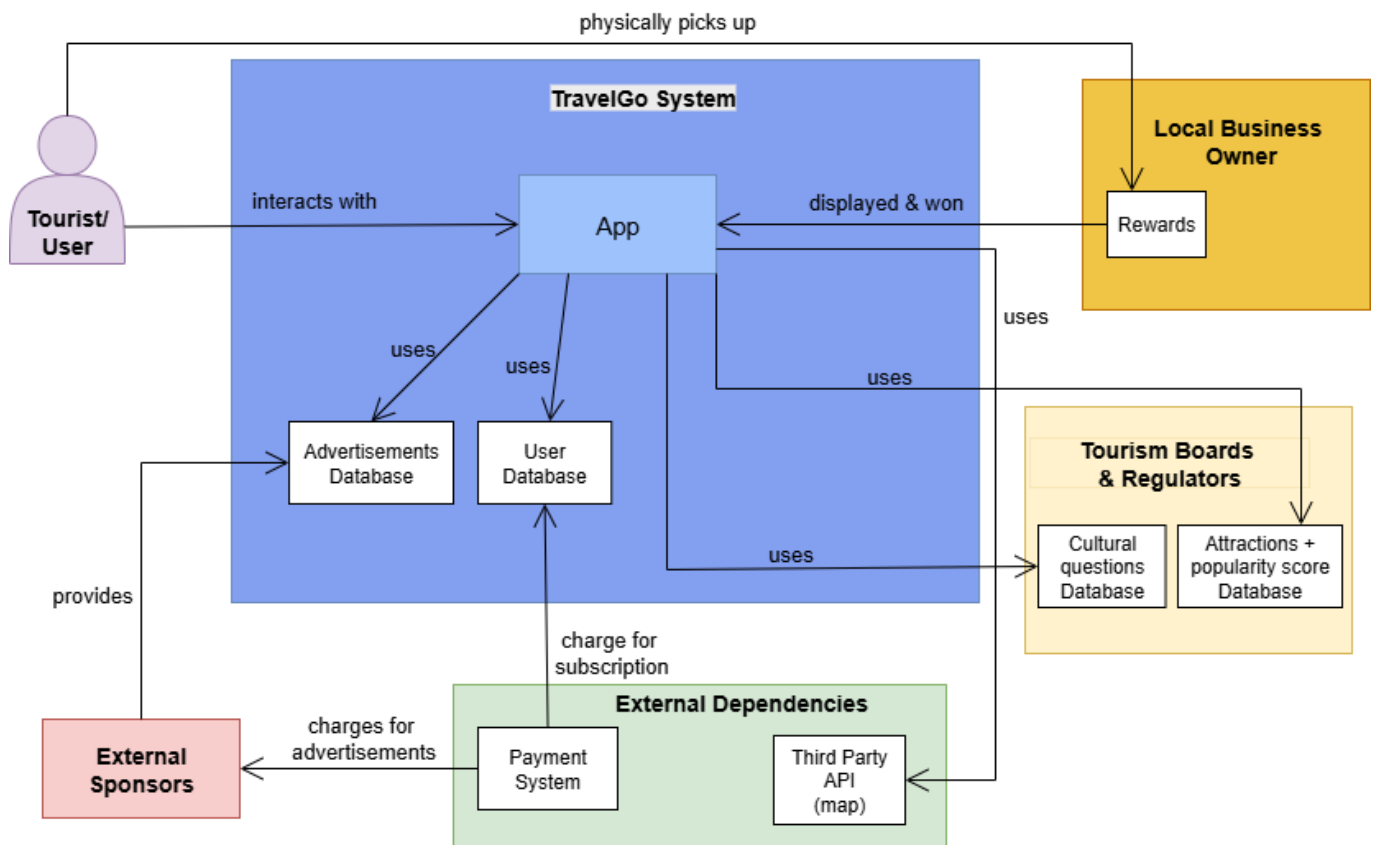


Figure 10.2.2: Container Diagram

10.2.3 Component View

The Component view bridges the gap between the system's high-level structure and its implementation details, gradually revealing how each container's functionality is realized. As can be seen from the Figure 10.2.3 below, the system's App will contain the presented components, which will later be described in the Code View.

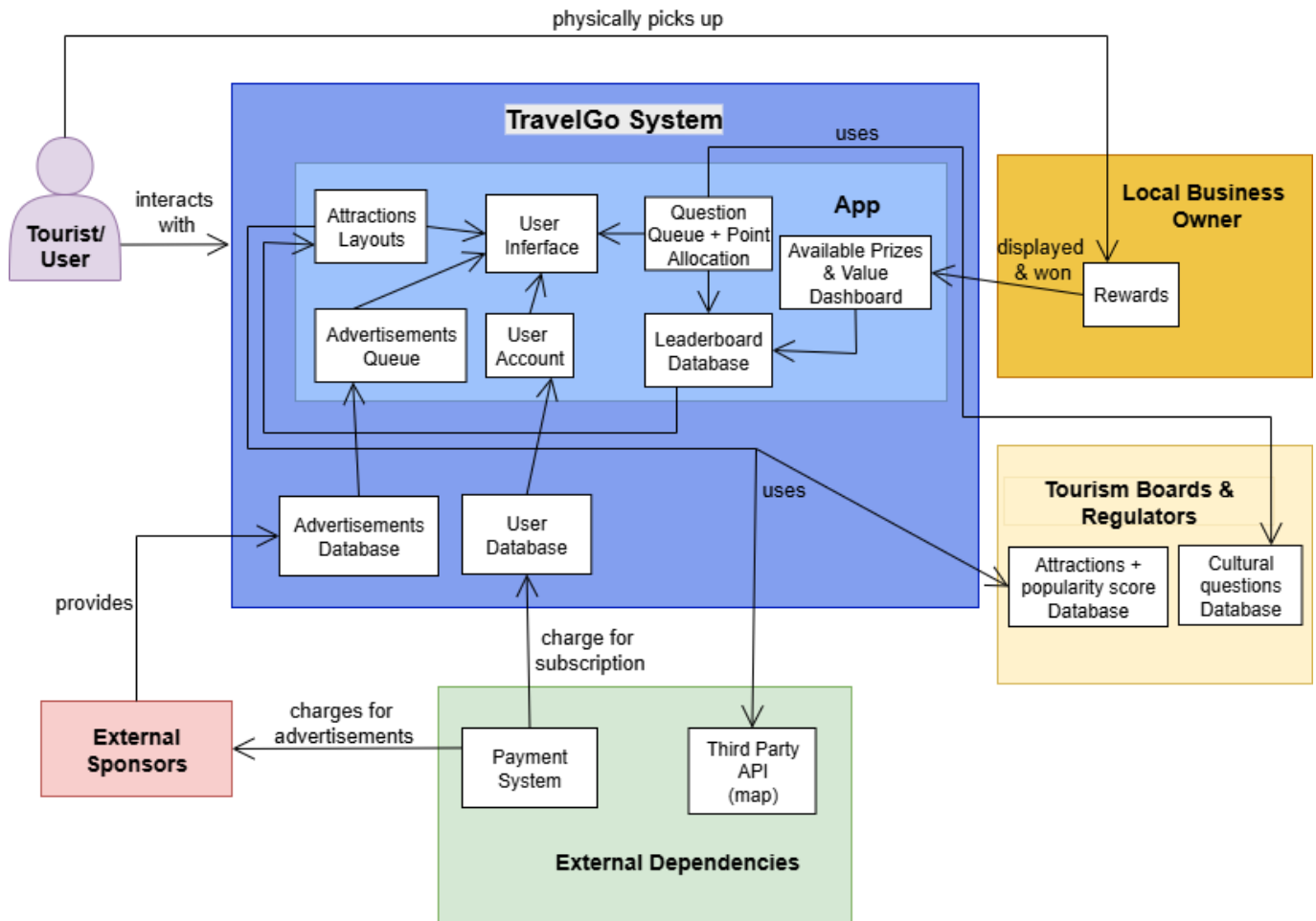


Figure 10.2.3: Component Diagram

10.2.4 Code View

10.3 Architectural & Design Patterns

Considering the proposed architecture and the proposed architectural structure of the TravelGo system, various architectural patterns would be suitable. In the following subsections we will present the primary patterns that should be implemented with respect to the most important quality attributes of the system. Additionally, an overview of all recommended patterns for the implementation of the system can be found in Appendix C

10.3.1 Event Driven

Scalability: Event-driven communication allows TravelGo to handle a large and diverse user base more efficiently. Instead of services constantly calling each other through direct APIs, they publish and subscribe to events via a broker. This reduces coupling and lets multiple services consume the same event without adding system strain. For instance, when a `QuestCompleted` event is published, the Leaderboard Service, Notification Service can all react independently. This enables TravelGo to scale individual services as demand grows, ensuring smooth performance during travel season spikes or viral content moments.

Modularity: TravelGo has distinct features such as the community chatroom, cultural side quests and league based competition, that all evolve at different speeds. Event-driven design supports loose coupling, meaning each service can be developed, deployed, and maintained independently. Adding new features is straightforward, a new service just subscribes to relevant events without disrupting existing ones. Introducing

a new module only requires subscribing to its related events, avoiding changes to other services.

Integrity: Events provide a structured, controlled way of sharing only the necessary data between services, improving data integrity and security. Sensitive data can be filtered at the broker, while services only receive the minimum required data needed, for instance, user IDs rather than full profiles. Moreover, event logs create an auditable trail of what happened and when, which strengthens TravelGo's reliability and accountability. If inconsistencies like leaderboard manipulation arise, events can be traced back to verify the source of truth.

10.3.2 CQRS (Command Query Responsibility Segregation)

Implementing CQRS would be beneficial for TravelGo, since it improves scalability, modularity and performance. Because the write side of the system would be separated from the read side, we can use a different model for the reads, which makes the read side a lot faster. This is advantageous since our system will have far more reads than writes, since our users will mainly use the map feature with all of the tourist attractions. It is also a lot easier to scale the read and write sides when they are independent from each other. Another benefit is that the data is easier to update, which improves the modularity of the system. This is especially important since we often have to add or change the data of the tourist attractions. Lastly, CQRS fits really well with event driven communication, especially event sourcing, which we plan to use.

10.3.3 SAGA Pattern

Although SAGA is a widely used design pattern for microservices, we do not believe it to be advantageous for our system. There are two main reasons for this. First of all, SAGA might cause increased latency, since services have to coordinate with each other. Because of this, the performance of our system might decrease. Second of all, SAGA is difficult to implement, and hard to debug. We do not believe that the advantages of SAGA outweigh these disadvantages.

10.3.4 API Gateway

The API Gateway acts as the central entry point for all client requests. Instead of the frontend or mobile application communicating directly with each individual microservice (like posts, chat, map, or leaderboard), all interactions first go through the API Gateway. It routes these requests to the correct service, aggregates data when needed, and returns a unified response to the user.

This design greatly simplifies communication between the frontend and backend systems. For example, when a traveller views the map, submits a post, or checks the leaderboard, the frontend sends requests only to the gateway. The gateway then coordinates with the relevant microservices such as the Map Service, Post Service, and Leaderboard Service, and compiles the response efficiently.

The API Gateway helps TravelGo scale horizontally by decoupling client interactions from the underlying microservices. Each service can be deployed, replicated, and scaled independently without affecting others. By isolating each service behind the API Gateway, TravelGo's architecture remains modular. All communication passes through the gateway as it is the central control point. The gateway can manage user authentication, enforce authorization, and apply HTTPS encryption to secure data in transit. This ensures that sensitive user information remains protected and that only authorized users can access specific features.

10.3.5 Circuit Breaker

In a distributed microservices system, the Circuit Breaker pattern acts like a protective mechanism ensuring that when a service begins failing or unresponsive, the circuit breaker disconnects it from the rest of the infrastructure and stops forwarding further calls. This approach is known as fast, but gracefully failing and it is preferred over waiting or retrying endlessly. While the circuit is open, requests immediately return an error or fallback without following the entire failing service. The pattern thus protects against cascading failures and keeps the resources from being exhausted early.

This pattern is a great fit since TravelGo depends on multiple remote services (maps, payments, rewards, tourism/attractions data, leaderboards) and it would mainly be used on the server side between services and third-party APIs. In the case of the rewards or payment systems, for example, if the services are down, the platform detects repeated failures and stops sending more requests to that service.

10.3.6 Retry Pattern

The Retry Pattern is a mechanism that automatically reattempts failed operations after a short delay, sometimes successfully helping the systems recover from temporary issues such as network timeouts. It would be suitable for our system as it will likely face multiple network or connectivity issues or brief spikes from the third-party APIs during high network traffic. Therefore, simply retrying after a short delay would often lead to the service succeeding in these situations.

11 Proof of Concept

The proof of concept demonstrates how the platform can bring together travellers and tourism industry workers in one ecosystem. It validates core features such as interactive maps, attraction discovery, user-generated content, and gamification elements like leaderboards. The POC was developed to demonstrate the technical feasibility of the platform's microservices-based architecture and to validate its core design principles; scalability, modularity, and reliability. The PoC serves as a minimal yet functional version of the TravelGo system, simulating the interaction between key components such as the map service, post service, leaderboard service, and chat service, all coordinated through an API Gateway and an event-driven communication model.

The implementation uses Python and Flask to represent each microservice, where every service runs independently on a separate port. This allows each module to be developed, deployed, and scaled independently; a direct demonstration of modularity and scalability. For instance, the post service handles user-generated travel experiences and attractions, while the leaderboard service listens for new posts via Kafka to dynamically update user scores. Similarly, the map service provides data on available destinations and attractions that appear on the home page.

At the center of the architecture lies the API Gateway, which acts as the single entry point for all client requests. It routes requests to the relevant services and aggregates responses where necessary. This setup not only simplifies communication between the frontend and backend but also improves system integrity by controlling and monitoring access to each microservice. The event-driven communication pattern, implemented through Kafka, allows services to communicate asynchronously. For example, when a new post is created, an event is broadcast to other services (the leaderboard), which can respond to it without being tightly coupled to the post service. This design ensures that each component can evolve or scale independently without disrupting the rest of the system.

The PoC includes a simple frontend interface built with Flask templates. The homepage displays an interactive map from the map service, while the posts, leaderboard, and chat pages represent user interaction points. Although simplified, this frontend demonstrates how user actions trigger backend operations and inter-service communication. The structure follows the directory layout of real-world modular systems, ensuring future compatibility with more advanced frameworks or containerized deployments (e.g., using Kubernetes).

11.1 External Dependencies

The platform relies on several external dependencies to function effectively. Some core services include maps and geolocation APIs (e.g., Google Maps) for navigation, routing, and location tracking. Furthermore, with respect to monetization, the platform depends on payment processors (IDEAL, PayPal, etc). On the technical side, the use of cloud hosting and databases would provide scalability and performance. Additionally, authentication services (Google, Facebook, etc) and communication tools (email/SMS providers) would also be employed to support user management.

For the current proof of concept implementation, we made use of [Leaflet](#) to develop the interactive map.

12 Revenue Model

In order to ensure long-term success for a platform, a sustainable revenue model is essential. The presented system would blend reality exploration with competitive gaming, meaning it can attract tourism-focused partnerships, as well as game-industry monetization.

12.1 Revenue Streams

The platform can be supported by income from multiple combined streams presented in the table below.

Revenue Stream	Implementation Details	Advantages	Challenges
Free Content / Subscription	Free tier with core features; premium tier unlocks exclusive content	Predictable recurring income; encourages retention	Requires adequate premium features to justify the cost
In-App Purchases	Cosmetic items, location-based boosts, hints, custom avatars	Transaction-based revenue resulting in immediate revenue from passionate users	High risk of warping user perception and turning the platform into "pay-to-win" if not balanced
Advertising & Sponsorships	Through advertisements, local businesses sponsor the discounts, souvenirs and/or events	Transaction-based revenue for non-premium users; Immediate income from sponsors	The advertising cannot be excessive in order to not degrade user experience

Table 12.1.1: List of viable revenue streams.

Furthermore, since the platform is newly developed, the revenue model should be implemented in progressive stages. At launch, most of the platform content should remain free to access to build the user base. Additionally, basic in-app purchases for cosmetic reasons can be included. During the next stage, the relation with local business owners would be established, and the platform would begin featuring sponsored

restaurants and souvenir shops, as well as custom maps and affiliation with tourist companies in the premium version. Lastly, the final stage could envision production of large-scale events, partnerships with museums from bigger cities and metropolises and potential merchandise sales.

12.2 Risks and Considerations

- User Experience: Excessive monetization risks pushing tourists away, therefore, the free version must remain engaging.
- Fairness: Competitive features must avoid "pay-to-win" dynamics.
- Scalability: Each additional revenue stream increases system complexity. As such, the platform should be built in a modular way so features can be added independently without interfering with existing ones.

13 Roadmap

The stages in which the proposed system will be implemented can be seen in the roadmap below.

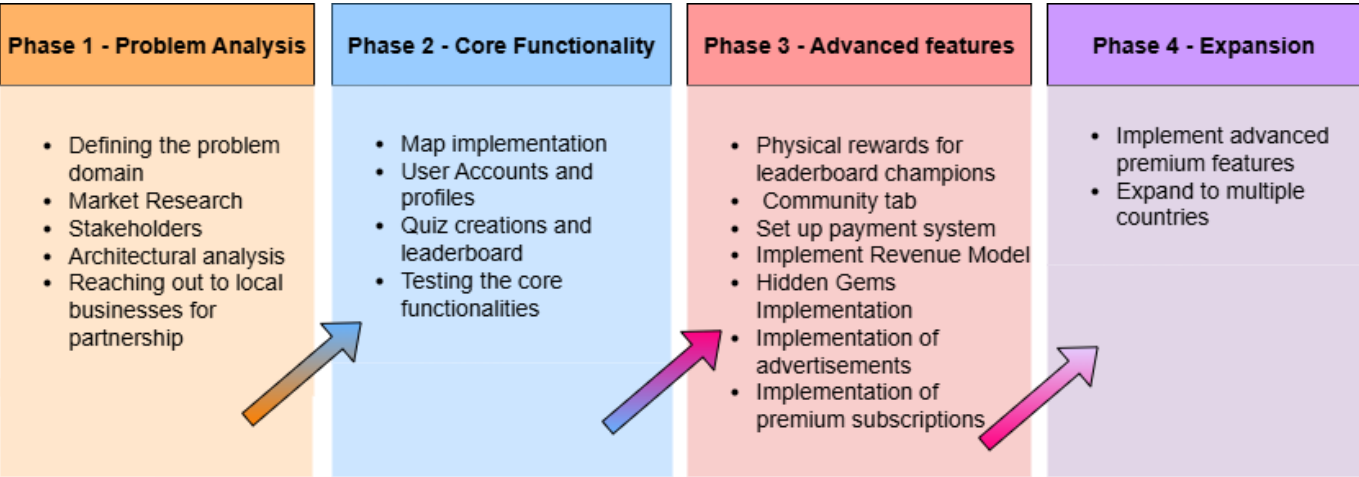


Figure 13.1: Roadmap

14 Open Source

To safe on time, cost and effort, we will make use of open source software. This does come with some risks: open source software might lead to compatibility issues, lack of support and potential security vulnerabilities. To mitigate these risks, all open source software must be carefully evaluated before usage. In some cases, open source tools might have to be adapted to fit our system better. For our POC, we already made use of several open source tools: more information can be found in section 11.1.

15 Discussion about Cloud

TravelGo has a global user base and considering it's microservices driven architecture, deploying the system on the cloud offers clear advantages in terms of scalability, flexibility, and integrity. The system aims to reach travelers, local communities, and businesses across the world. Therefore, leveraging cloud infrastructure ensures low latency and seamless accessibility across regions. Cloud providers offer geographically distributed data centers, enabling TravelGo to host its services closer to its users and deliver fast, consistent performance even during high-traffic periods such as holiday seasons or major events.

A public cloud environment such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) will be a good fit for the system. It provides cost efficiency, elasticity, and rapid deployment without the heavy maintenance required for private infrastructure. However, certain sensitive components, such as payment systems and user data, could also benefit from a hybrid approach, where confidential data is stored in a private cloud or restricted region to comply with regulations such as GDPR, while general application services operate on the public cloud.

In terms of service models, TravelGo would best utilize Platform as a Service (PaaS) or container-based solutions like AWS ECS or Google Kubernetes Engine. These services abstract away low-level infrastructure management while supporting independent deployment of microservices.[6] This complements TravelGo's modular design where each service can be deployed and scaled separately depending on demand. Serverless offerings, such as AWS Lambda or Google Cloud Functions, could also support event-driven features like notifications or leaderboard updates, providing efficient scalability for unpredictable workloads.

Cloud deployment directly supports TravelGo's key quality attributes by enabling scalability, modularity, and integrity. Through auto-scaling mechanisms, the platform can dynamically adjust resources based on user demand. [7] Each microservice can be containerized and managed independently, allowing for isolated updates and seamless feature expansion. Moreover, built-in cloud security measures such as encryption, identity and access management (IAM), and DDoS protection enhance system reliability and secure user data.

Despite its advantages, cloud deployment introduces certain challenges. Vendor lock-in may restrict flexibility in switching providers, while data residency laws, particularly in the EU, require careful configuration to ensure compliance.[8] Furthermore, managing a distributed microservices environment in the cloud adds operational complexity. These challenges can be mitigated through the use of cloud-agnostic technologies such as Docker and Kubernetes, and region-specific deployments for user data.

In conclusion, deploying TravelGo on the cloud complements its microservices based design by providing global scalability, modular growth, and secure operations. The cloud not only supports the system's technical requirements but also aligns with its vision of delivering a smooth, engaging, and reliable travel experience to users worldwide.

Bibliography

- [1] Pautasso, C. (2020). Software Architecture: visual lecture notes. LeanPub. <https://leanpub.com/software-architecture/>
- [2] Alčaković, S., Pavlović, D., & Popesku, J. (2017). Millennials and gamification: A model proposal for gamification application in tourism destination. *Marketing*, 48(4), 207–214. <https://doi.org/10.5937/markt1704207a>
- [3] Gen Z Travel Trends: Statistics, Insights and what it all means for the industry [2025]. (n.d.). Atlys. <https://www.atlys.com/blog/gen-z-travel-trends>
- [4] Pitrelli, M. (2023, March 27). More millennials are turning 40 — and they're changing travel as we know it. CNBC. <https://www.cnbc.com/2023/03/27/millennials-are-turning-40-and-theyre-changing-travel-as-we-know-it.html>
- [5] Artug, E., & Fateh, D. (2025, March 28). Serverless and microservices: A tale of two architectures. Contentful. <https://www.contentful.com/blog/serverless-vs-microservices/> (Date Accessed - October 2025)
- [6] Types of Cloud Computing. AWS. <https://aws.amazon.com/types-of-cloud-computing/>
- [7] Cody Slingerland. (2023). What Is Cloud Scalability? Benefits And Tips For Every Organization. CloudZero. <https://www.cloudzero.com/blog/cloud-scalability/>

- [8] Chrystal R. China, & Michael Goodwin (2025). IaaS, PaaS, SaaS: What's the difference? IBM. <https://www.ibm.com/think/topics/iaas-paas-saas>
- [9] Brown, S. (n.d.). The C4 model for visualising software architecture. C4 Model. Retrieved October 12, 2025, from <https://c4model.com/> (Date Accessed - October 2025)
- [10] Ahmad, A. (2025, August 23). 19 Essential Microservices Patterns for System Design Interviews. Design Gurus. https://www.designgurus.io/blog/19-essential-microservices-patterns-for-system-design-interviews?gad_source=1&gad_campaignid=21052024757&gbraid=0AAAAADME9yrt3rLA-YSrKYswgzdQyBX6D&gclid=Cj0KCQjwovPGBhDxARIsAFhgkwRavr_Fn1z55RbDBcbpqNeaZ_L5WuzKZd0gBhH05Vf0RLmLTqh8ahUaAkCBEALw_wcB (Date Accessed - October 2025)
- [11] Sethi, R. (2022). 3.1.3 Kinds of Requirements [ISBN 9781316511947]. In Software Engineering: Basic Principles and Best Practices. Cambridge University Press (1st ed., pp. 181-185).